C PROGRAMMING

Lesson 2 — Variables & Data Types

Based on: Bro Code C Tutorial Series

1. What is a Variable?

A variable is a named container that stores a value in memory. It behaves as if it were that value, meaning you can use the variable name anywhere you would use the actual value.

Every variable in C has:

- **A data type —** what kind of value it stores (number, character, etc.)
- **A name —** how you refer to it in your code
- **A value —** the actual data stored inside it

```
int age = 25;   // type: int | name: age | value: 25
```

> **Key Rule:** Every variable statement must end with a semicolon ;

2. Data Types

C requires you to declare the type of every variable before using it. The type tells the compiler what kind of data the variable holds and how much memory to allocate.

int — Whole Numbers

Stores whole numbers only. Decimal parts are truncated (cut off), not rounded.

```
int age = 25;
int year = 2024;
printf("Age: %d\n", age);   // format specifier: %d
```

- **Format specifier:** %d
- **Memory:** 4 bytes
- **Use for:** age, year, quantity, count

> **Watch Out:** If you assign 2.9 to an int, it stores 2 — the decimal is cut off, not rounded.

float — Decimal Numbers (Single Precision)

Stores decimal numbers with about 6-7 digits of precision.

```
float gpa = 3.75;
float price = 9.99;
printf("GPA: %f\n", gpa);   // format specifier: %f
```

- **Format specifier:** %f
- **Memory:** 4 bytes
- **Use for:** GPA, price, temperature

> **Note:** By default, %f prints 6 decimal places. Precision control will be covered in a future lesson.

double — Decimal Numbers (Double Precision)

Like float, but stores up to 15-16 digits of precision. Use this when accuracy really matters.

```
double pi = 3.14159265358979;
printf("Pi: %lf\n", pi);   // format specifier: %lf
```

- **Format specifier:** %lf
- **Memory:** 8 bytes
- **Use for:** pi, scientific constants, Euler's number

char — Single Character

Stores exactly one character. Must use single quotes ' ' not double quotes.

```
char grade = 'A';
char symbol = '$';
printf("Grade: %c\n", grade);   // format specifier: %c
```

- **Format specifier:** %c
- **Memory:** 1 byte
- **Use for:** grades, symbols, currency signs

> **Important:** Single quotes ' ' for char. Double quotes " " for strings. Mixing them causes errors.

char[] — String (Array of Characters)

C has no built-in string type. Instead, strings are arrays of characters. Use double quotes " ".

```
char name[] = "Bro Code";
char email[] = "bro@gmail.com";
printf("Name: %s\n", name);   // format specifier: %s
```

- **Format specifier:** %s
- **Memory:** varies (depends on string length)
- **Use for:** names, emails, words, sentences

> **Note:** Strings can contain numbers ("123") but they behave as text, not numeric values.

bool — Boolean (True / False)

Stores only two values: true or false. Requires an extra header file to use.

```
#include <stdbool.h>   // required for bool
```

```
bool isOnline = true;
bool hasAccount = false;
printf("%d\n", isOnline);   // prints 1 (true) or 0 (false)
```

- **Format specifier:** %d (prints 1 for true, 0 for false)
- **Memory:** 1 byte
- **Use for:** conditions, flags, on/off states

> **Remember:** Booleans are mainly used internally for logic (if statements, loops). You rarely print them directly.

3. Practice Problem

Practice: Student Profile

Create a C program that stores and prints a student profile using all 6 data types covered in this lesson.

Requirements:

- Declare one variable of each type: int, float, double, char, char[], bool
- Use meaningful names (e.g. age, gpa, pi, grade, name, isEnrolled)
- Print each variable with printf() using the correct format specifier
- Add a comment next to each variable explaining what it stores

Expected Output (example)

```
Name: Alex
Age: 17
GPA: 3.750000
Grade: A
Enrolled: 1
```